Agora — Phased Implementation Plan

> Derived from PRD v1.0 and Architecture Design Document Each phase builds on the previous, introducing new platform capabilities incrementally. Key principle: **validate the platform core with the simplest mode first, then add complexity.**

Phase Overview

Phase	Name	Status	Core Unlocks	New Modes
1	Roundtable MVP	✅ **DONE**	Agent, Room, RoundRobin Flow, LLM multi-provider, basic UI	Roundtable Debate
2a	Werewolf Core	✅ **DONE**	Channel isolation, StateMachine Flow, Structured Output	Werewolf (5 roles)
2b	Werewolf Advanced	✅ **DONE**	Togglable advanced rules	Werewolf (7 roles: +Guard, +Idiot)
3	Frontend + Observability	✅ **DONE**	Mode-specific UI, Token/cost tracking (LiteLLM), Observability timeline	— (enhance existing)
4	Persistence + Replay	✅ **DONE**	Postgres (Supabase) event store, DB-backed reads, /replays + /replay/[id] with animated playback, Vercel-ready runtime	— (enhance existing)
4.5a	Durable Runtime (AI-only)	✅ **DONE**	Bespoke advanceRoom + chained tick dispatcher, deterministic agent IDs, waiting state, /admin observability, replay determinism	— (foundation; AI-only games complete reliably)
5	UI Overhaul (i18n + Round Table)	✅ **DONE**	next-intl (en/zh), round-table viz, click-to-view modal, WeChat chat sidebar, Accio palette, chat-default view	— (visual polish)
6	Team Platform (the real Agora)	✅ **DONE** **2026-04-15**	Five primitives (agents · teams · rooms · modes · templates), 4 ship-with templates, open-chat mode, DiceBear avatars, AppShell + Sidebar,	Open-chat

Phase	Name	Status	Core Unlocks	New Modes
			team-snapshot room creation, leader directive via prompt-append	
4.5b	Human-Play UX Design	✅ **DONE 2026-04-15**	V2 spec: 15 sections, 12 turn panels, visibility matrix, timer/fallback/disconnect UX, component inventory. Self-critiqued + audit-verified (12 issues fixed).	— (doc deliverable: `docs/design/phase-4.5b-human-play-ux.md`)
4.5c	Seat Tokens + Human Play	✅ **DONE 2026-04-16**	HumanAgent, seat token (localStorage for MVP), human-input POST route, 7 werewolf panels (Vote/Witch/Seer/Guard/Hunter/Sheriff election+transfer), "Play as" dropdown, HumanPlayBar with phase dispatch. Open-chat + werewolf supported. E2E verified. Roundtable deferred (still on legacy waitUntil path).	1-human-N-AI games
Design migration	Linear-spec design system	✅ **DONE 2026-04-17**	Inter typography, Linear-derived dark-mode tokens, Agora mint accent, full UI migration (sidebar, cards, hero, chat bubbles), WCAG contrast pass. Commits `3e0bd0c → 0b1541f` .	— (visual polish)
AI SDK v6 upgrade	Dependency baseline	✅ **DONE 2026-04-17**	`ai@4 → ai@6` , `@ai-sdk/* 1.x → 3.x` , token extraction + fallback pricing fix, `temperature` param strip. Commits `5cfa662 → 0ab76b5` .	— (foundation)
4.5d	Multi-Human + Supabase Auth	🔶 **IN PROGRESS**	Auth foundation + JWT invites + multi-human picker shipped (`c858213` , `5b73b6d`). Presence/disconnection/fan-in remaining. **Triggers workflow-primitive re-eval** per `docs/design/workflow-architecture.md`	N-human rooms
7	TRPG	⏸ After 4.5d	GM Agent, Dice system, Narrative generation, Character growth. Likely **triggers WDK migration** for durable long pauses	TRPG

Phase	Name	Status	Core Unlocks	New Modes
8	Script Kill	⏸ Later	Long-term Memory (pgvector), Clue/Evidence system, Branching Narrative	Script Kill
9	Platform / Custom Mode SDK	⏸ Later	Custom Mode SDK, Agent Marketplace, Replay sharing, Hierarchical Flow	Custom

Plan: `docs/design/phase-6-team-platform.md` · **Handoff**: `docs/design/phase-6-handoff.md` · **Workflow**: `docs/design/workflow-architecture.md`

Current session date: 2026-04-28 **Phase 6 shipped in commit**: `f5d71c5` → `https://agora-panpanmao.vercel.app` **Active phase**: 4.5d — multi-human runtime (presence + fan-in + disconnect) **Architectural decision (2026-04-28)**: 4.5d-2 (parallel fan-in) and Phase 7 long-pauses will migrate to **Vercel Workflow DevKit**. WDK reached GA on 2026-04-16, retiring the "wait for settled API surface" condition from the 2026-04-15 review. See `docs/design/workflow-architecture.md` § 2026-04-28 update.

Phase 1: Roundtable Debate — Validate Platform Core

Goal: N agents (each using a different LLM model) debate a user-given topic in a group chat UI. Proves that Room, Agent, FlowController, and multi-model LLM integration work end-to-end.

Step 1.1: Project Scaffolding

- [ ] Initialize Turborepo monorepo
- [ ] Create `apps/web` with Next.js 15 (App Router)
- [ ] Create `packages/core` (empty, with tsconfig)
- [ ] Create `packages/llm` (empty, with tsconfig)
- [ ] Create `packages/modes` (empty, with tsconfig)
- [ ] Create `packages/shared` (types, constants)
- [ ] Configure Tailwind + shadcn/ui in `apps/web`
- [ ] Verify `turbo dev` runs the frontend

Deliverable: `turbo dev` serves a blank Next.js app.

Step 1.2: Core Abstractions (packages/core)

- [ ] `Message` type — id, sender, content, channel, timestamp, metadata
- [ ] `Agent` interface — reply(), observe(), persona config, model binding
- [ ] `AIAgent` implementation — wraps Vercel AI SDK `generateText`

- [ ] `Room` class — create, add/remove agents, lifecycle (waiting → active → ended)
- [ ] `FreeFormFlow` — simplest FlowController: round-robin with optional max rounds
- [ ] `EventBus` — typed event emitter for UI subscription (messageCreated, agentReplied, roundChanged)

Deliverable: Can create a Room, add 3 AIAgents, run a FreeFormFlow, and see messages in console.

Step 1.3: LLM Integration (packages/llm)

- [ ] Vercel AI SDK wrapper — `createAgentModel(provider, modelId, apiKey?)`
- [ ] Multi-provider support: Anthropic (Claude), OpenAI (GPT), Google (Gemini)
- [ ] Streaming support via `streamText`
- [ ] Error handling + retry logic
- [ ] API key management (env vars for MVP)

Deliverable: Can call Claude, GPT, and Gemini through a unified interface.

Step 1.4: Roundtable Mode (packages/modes/roundtable)

- [ ] Mode definition: roles = [{id: "debater", count: [2, 8]}]
- [ ] Flow config: FreeFormFlow with configurable rounds (default 3-5)
- [ ] System prompts: debater persona template
- [ ] Optional: vote for best argument at end of each round
- [ ] Moderator logic: announce topic, call each agent in turn, summarize

Deliverable: Roundtable mode registered and runnable via Room.

Step 1.5: Frontend — Basic Chat UI (apps/web)

- [ ] Home page: "Create Room" button
- [ ] Room creation form: topic, number of agents, model selection per agent
- [ ] Room page: group chat UI with message bubbles
- Agent name + model badge on each bubble
- Different colors per agent
- Auto-scroll, typing indicator
- Markdown rendering in messages
- [ ] API routes: POST /api/rooms, GET /api/rooms/:id, POST /api/rooms/:id/start
- [ ] Socket.io integration: real-time message streaming from server to client

Deliverable: User creates a room in browser, picks a topic and 3-5 agents with different models, hits "Start", and watches them debate in real-time.

Step 1.6: First Deploy

- [ ] Deploy to Vercel
- [ ] Environment variables configured (API keys)

- [ ] Verify end-to-end flow works in production

Acceptance Criteria (Phase 1): - [ ] User can create a room with a topic - [ ] User can add 2-8 AI agents, each with a different model - [ ] Agents debate the topic in a group chat, each maintaining their persona - [ ] Messages stream in real-time to the browser - [ ] Each agent's model is visually indicated (badge) - [ ] A debate completes 3-5 rounds without errors - [ ] Deployed and accessible via public URL

Phase 2: Werewolf — Channel Isolation + State Machine

Goal: Full werewolf game with night/day phases, private wolf channel, voting, and elimination. Proves Channel isolation and StateMachine flow work.

Step 2.1: Channel System (packages/core)

- [ ] `Channel` class — id, subscribers, autobroadcast toggle
- [ ] Nested channels — outer (all players) + inner (wolf-only)
- [ ] Visibility mask — message routing respects channel membership
- [ ] Dynamic subscribe/unsubscribe (dead players removed)
- [ ] Channel-aware EventBus — only emit to clients who should see

Step 2.2: StateMachine FlowController (packages/core)

- [ ] `StateMachineFlow` — generic state machine with phases, transitions, actions
- [ ] Phase definition: { id, allowedChannels, actions, nextPhase, condition }
- [ ] Action types: discuss (sequential), vote (fanout), announce (broadcast)
- [ ] Transition conditions: vote result, timer, custom predicate

Step 2.3: Structured Output (packages/core)

- [ ] `createDecisionSchema(type, context)` — generates Zod schema dynamically
- [ ] Vote schema: `z.object({ target: z.enum(alivePlayers) })`
- [ ] Discussion schema: `z.object({ message: z.string(), reachAgreement: z.boolean() })`
- [ ] Vercel AI SDK `generateObject` integration for constrained decisions

Step 2.4: Werewolf Mode (packages/modes/werewolf)

- [ ] Roles: werewolf (2-3), villager (3-4), seer (1), witch (1)
- [ ] Phases: Night (wolf discuss → wolf vote → seer check → witch act) → Day (announce → discuss → vote → eliminate)
- [ ] Channel config: `public` (day), `werewolf` (night), `seer-result` (private), `witch-action` (private)
- [ ] Win conditions: all wolves dead OR wolves >= villagers
- [ ] Role assignment: random shuffle at game start
- [ ] Detailed prompts per role (wolf strategy, seer investigation, witch dilemma)

Step 2.5: Frontend — Game UI

- [ ] Phase indicator (night/day with visual transition)
- [ ] Vote panel (shows alive players, collects votes)
- [ ] Role reveal (private, only shown to the agent's owner)
- [ ] Elimination announcement with animation
- [ ] Win/loss screen
- [ ] Night overlay (dim screen during wolf phase if spectator)

Acceptance Criteria (Phase 2): - [ ] 6-9 agent werewolf game runs to completion - [ ] Wolves can discuss privately — villagers NEVER see wolf chat - [ ] Seer gets private check result — no one else sees it - [ ] Votes are collected independently (no one sees others' votes before reveal) - [ ] Win condition is correctly detected and game ends - [ ] Spectator can see everything (god mode toggle) - [ ] Adding werewolf mode did NOT require changes to packages/core

Phase 3: Frontend + Observability + Token Tracking ✅ DONE

Goal achieved: Platform fully accessible via UI, with live token cost tracking and observability timeline.

Handoff doc: `docs/design/phase-3-handoff.md` (historical — see commit log 748d9c4, 90e2ad6, 526618d, b268897)

Step 3.1: Token Usage + Cost Tracking ✅ (commit 748d9c4)

- [x] `packages/llm/src/pricing.ts` — LiteLLM registry with offline fallback
- [x] `packages/shared/src/types.ts` — `TokenUsage`, `ModelPricing`, `TokenUsageRecord` + `token:recorded` event
- [x] `packages/llm/src/generate.ts` — `{ content, usage }` return, extracts Anthropic cache + OpenAI reasoning metadata
- [x] `packages/core/src/token-accountant.ts` — aggregates per agent/model/room, emits token:recorded
- [x] `packages/core/src/agent.ts` — usage + provider + modelId land in Message.metadata
- [x] `scripts/token-report.ts` — live LiteLLM pricing report
- [x] `scripts/run-werewolf.ts` — prints + embeds summary in transcript

Validated: 6-player game, 43 calls, 110k tokens, $0.4340 — accurate per-model cost breakdown.

Step 3.2: Generic Frontend Components ✅ (commit 90e2ad6)

- [x] `apps/web/app/room/[id]/components/theme.ts` — palette + types + helpers
- [x] `apps/web/app/room/[id]/components/MessageList.tsx`
- [x] `apps/web/app/room/[id]/components/AgentList.tsx`
- [x] `apps/web/app/room/[id]/components/ChannelTabs.tsx`

- [x] `apps/web/app/room/[id]/components/PhaseIndicator.tsx`
- [x] `apps/web/app/room/[id]/components/TokenCostPanel.tsx` — collapsible, per-model + per-agent
- [x] `apps/web/app/room/[id]/hooks/useRoomPoll.ts` — shared polling hook
- [x] `apps/web/app/room/[id]/page.tsx` — thin dispatcher by `modeId`
- [x] `apps/web/app/api/rooms/[id]/messages/route.ts` — returns `tokenSummary` + `modeId` + `currentPhase`

Step 3.3: Roundtable Refactor ✅ (commit 90e2ad6)

- [x] `apps/web/app/room/[id]/modes/roundtable/RoundtableView.tsx`

Step 3.4: Werewolf Frontend ✅ (commit 526618d)

- [x] `apps/web/app/create-werewolf/page.tsx` — player count, model per slot, rule toggles
- [x] `apps/web/app/api/rooms/werewolf/route.ts` — wires `createWerewolf` + accountant + gameState snapshot
- [x] `apps/web/app/room/[id]/modes/werewolf/WerewolfView.tsx` — phase banner, channel tabs, role badges, night gradient, winner banner
- [x] New landing page with mode cards (Debate + Werewolf)
- [x] Role emoji + alive/dead state in agent pills
- [ ] NightOverlay / VoteSummary — deferred (basic night-gradient works; animation polish later)
- [ ] Player perspective switcher — deferred (spectator sees all currently)

Step 3.5: Observability ✅ (commit b268897)

- [x] `apps/web/app/api/rooms/[id]/events/route.ts` — indexed events with ?after= paging
- [x] Both room routes persist `token:recorded` in the event log
- [x] `apps/web/app/room/[id]/components/Timeline.tsx` — filterable by type + agent, color-coded
- [x] `apps/web/app/room/[id]/observability/page.tsx` — parallel poll events + messages
- [x] "Timeline →" deep link in both room views
- [ ] `AgentMemoryInspector` — deferred (event timeline covers most debugging needs)
- [ ] `DecisionTree` — deferred (decisions already surface in MessageList with JSON pretty-print)

Step 3.6 (Optional): Testing Infrastructure

- [ ] Deferred to Phase 4

Acceptance Criteria (Phase 3): - [x] Werewolf game fully playable through UI — /create-werewolf → /room/[id] - [x] Live token cost visible during any game (collapsible panel in room header) - [x] Observability page shows event timeline with filters - [x] Mode-specific views share common components (theme.ts + components/*) - [ ] Spectator player-perspective switcher — deferred (all-roles-visible is acceptable for demos) - [ ] Deploy to Vercel public demo — deferred (next milestone)

Deferred from Phase 3 (originally "UX Polish")

These deferred to Phase 3.5 or later — good post-Phase-3 work once the core UI is solid:

- [ ] 2D room visualization with agent avatars positioned around a table
- [ ] Framer Motion animations for speaking/voting/elimination
- [ ] Persona editor with AI auto-enrich
- [ ] Persona library (save/reuse across rooms)
- [ ] Human player support (`HumanAgent` in packages/core)
- [ ] Mixed rooms (AI + human)

Phase 4: Persistence Foundation + Replay ✅ DONE

Shipped 2026-04-14. See `docs/design/phase-4-plan.md` for the full rationale.

Commits on main: - **ba4986d** — Phase 4 plan doc - **52e05f8** — packages/db with Drizzle + schema + migration + Supabase hookup - **52e05f8** — room-store rewrite: Postgres reads, waitUntil-based game runtime, event-log write-through - **8dec1c1** — /replays + animated /replay/[id] with scrubber + speed controls

Validated locally: debate + werewolf games persist across server restarts; replays reconstruct full UI state from events.

Phase 4.5: Durable Runtime + Human-in-the-Loop (V2, reordered around Phase 5)

Goal: Replace `waitUntil()` -bundled game loops with a bespoke ~500 LOC durable runtime (`advanceRoom` + chained `/api/rooms/tick` invocations). Add Agent abstraction (AI + Human), seat-token MVP auth, Supabase Realtime, and ultimately Supabase Auth. Mixed human+AI rooms with N humans supported architecturally.

Triggered by: 3 of 6 zh seed werewolf games orphaned at the 5-min wall. Proper fix, not stopgap.

Plan: `docs/design/phase-4.5-plan.md` — V2 design after self-critique of initial V1 (V1 over-engineered a generic workflow engine; V2 is bespoke + reordered).

Key architectural choices: - Bespoke runtime in `apps/web/app/lib/room-runtime.ts` , NOT a generic workflow engine - Event sourcing IS the step log (no new `workflow_*` tables) - Determinism via seeded agent IDs + seeded shuffle in `createWerewolf` - Seat tokens (signed JWT bound to `room_id` + `agent_seat_id`) as MVP auth — zero friction for zh users, no email/OAuth dependency - Supabase Auth arrives in 4.5d as a layer atop seat tokens (persistent identity) - TDD for replay determinism; `/admin/rooms/:id` for observability from day 1

4.5a: AI-only durable runtime (~4 days) — unblocks Phase 5

- [ ] Migration: `rooms.waiting_for`, `rooms.waiting_until`, status CHECK adds `'waiting'`
- [ ] `advanceRoom(roomId)`, `loadRoomState`, state folding helpers in `apps/web/app/lib/room-runtime.ts`
- [ ] Deterministic refactor: `createWerewolf` takes pre-generated agent IDs + seed
- [ ] Mode-specific `advance.ts` files in `packages/modes/<mode>/`
- [ ] `/api/rooms/tick` dispatcher + inline self-invoke chain; pg_cron every 5s as safety net
- [ ] `/admin/rooms/:id` observability view (phase timeline, waiting state)
- [ ] TDD: replay determinism test suite
- [ ] Remove `waitUntil` game-loop bundling from /api/rooms + /api/rooms/werewolf
- [ ] **Exit**: 12p AI-only werewolf completes on Vercel prod; replay determinism verified

(Phase 5: UI overhaul — 5.2-5.6 ~10-12 days) — unblocked after 4.5a

Runs in parallel-ish with itself, see `docs/design/phase-5-plan.md`. Human seats render identical to AI seats; kind-aware affordances land in 4.5c.

4.5b: Human-play UX design spec (~2 days, design-only, no code)

- [ ] `docs/design/phase-4.5b-human-ux.md`: wireframes + copy for each turn type (vote, witch, seer, speak, last-words)
- [ ] Info visibility matrix per role × phase
- [ ] Microcopy for prompts, timeouts, fallbacks
- [ ] Disconnection UX
- [ ] V1 → self-critique → V2 iteration
- [ ] **Exit**: user signs off on the UX spec before any code

4.5c: Seat tokens + human play implementation (~4-5 days)

- [ ] `packages/core/src/agent.ts`: `Agent` interface + `HumanAgent` implementation
- [ ] `apps/web/app/lib/seat-tokens.ts`: JWT signing/verification
- [ ] `POST /api/rooms/:id/invites`: owner generates N seat-bound invite URLs
- [ ] `GET /r/:roomId?seat=X&token=Y`: token landing, stores in localStorage
- [ ] `POST /api/rooms/:id/human-input`: token-gated, inserts `'human:input'` event
- [ ] `useRoomLive` hook: Supabase Realtime subscription + polling fallback
- [ ] `ViewerContext`, `MyInputPanel`, `SchemaForm` components
- [ ] Server-side channel filter in `/api/rooms/:id/messages`
- [ ] Per-mode timeout policies + fallbacks
- [ ] **Exit**: 1-human-8-AI werewolf game completes end-to-end with human witch

4.5d: Multi-human + Supabase Auth layer (IN PROGRESS — ~5-7 days remaining)

Shipped (commits `c858213`, `5b73b6d`, `c01119c`): - [x] Supabase Auth — magic-link signup; `/auth/callback` route checks `allowed_emails` allowlist before granting session - [x] `allowed_emails` table + RLS (migration `0007_allowed_emails.sql`) - [x] JWT seat invites — `POST /api/rooms/[id]/invites`, multi-human picker UI on room creation - [x] Mid-phase resume bugfix — replay routes through `flow.onMessage` so chained-tick rehydration is correct after a human turn (commit `c01119c`)

Remaining (split into shippable sub-phases):

4.5d-1 — Presence + disconnection grace (~2 days, Tier 3 GSD-lite)

- [ ] Supabase Realtime presence channel per room (subscribe on room mount, broadcast heartbeat)
- [ ] `useRoomLive` hook: Realtime subscription with polling fallback
- [ ] 30s disconnection grace timer per human seat; UI heartbeat indicator
- [ ] Server-side: on grace expiry, advance via mode's fallback policy (auto-vote, skip, etc.)
- [ ] **Exit**: human disconnect during day-vote falls back to mode default within 30s; reconnect within grace resumes seat

4.5d-2 — Parallel fan-in via WDK (~3 days, Tier 4 — architectural)

- [ ] Add `workflow` package, configure WDK runtime in `apps/web`
- [ ] Wrap `advanceRoom` in a workflow function — phases become `step.run()` boundaries
- [ ] Day-vote fan-in: `Promise.race([Promise.all(humanVoteHooks), sleep('60s')])` pattern; iterable `createHook<Vote>()` per human seat
- [ ] Coexistence contract: WDK calls `flow.onMessage` so the `events` table stays source of truth for replay (do NOT replace event-log replay with WDK's internal log)
- [ ] Feature-flag the WDK path; keep HTTP-chain path for one week as rollback
- [ ] **Exit**: 2+ humans vote in parallel during werewolf day-phase; timer-only fallback fires correctly when not all humans respond

4.5d-3 — Multi-human exit verification (~1-2 days, Tier 3)

- [ ] 2-human-7-AI werewolf E2E playthrough (manual + scripted)
- [ ] Disconnection recovery test: kill one tab mid-game, verify fallback + reconnect
- [ ] Roundtable durable-runtime migration (still on legacy `waitUntil` per 4.5c notes — fold in here while runtime is being touched)
- [ ] Update `docs/design/phase-4.5-plan.md` with as-built notes
- [ ] **Exit**: 2-human-7-AI werewolf completes including day-vote fan-in + disconnection recovery (original 4.5d exit criterion)

Note on `room_memberships`: original plan called for a `room_memberships` table + RLS. Current architecture skips this — JWT seat tokens encode room+seat scope and the `allowed_emails` gate handles signup access control. Re-evaluate once 4.5d-1 ships if presence needs persistent membership rows; otherwise leave on tokens.

Total revised: ~27 days across 5 ship-able milestones. UI overhaul shipped ~day 16 (on track). 4.5d completion ~day 24 with WDK migration; was ~day 27 with hand-rolled fan-in.

Phase 5: UI Overhaul — i18n + Round-Table Visualization

Goal: Replace the message-list UI with a round-table layout (agents in a circle, speech bubbles above each seat), WeChat-style chat sidebar, and click-to-view agent details modal. Add en/zh i18n so the platform works for Chinese users.

Plan: `docs/design/phase-5-plan.md` · **Handoff**: `docs/design/phase-5-handoff.md`

5.1: i18n foundation + agent-language directive ✅ DONE (commits c158eca, 446677e)

- [x] next-intl wired (cookie-based locale, no URL prefix)
- [x] Dictionaries `apps/web/messages/{en,zh}.json` (8 namespaces)
- [x] LocaleSwitcher component + `/api/locale` route
- [x] All pages translated (landing, create, create-werewolf, replays, replay, room, observability, all components)
- [x] Agent-language directive: `/api/rooms` and `/api/rooms/werewolf` accept `language: 'en'|'zh'`; falls back to cookie then UI locale. `createWerewolf` accepts `languageInstruction`.
- [x] Frontend create pages send `language: useLocale()` so agents default to UI language
- [x] 6 zh demo replays seeded via `scripts/seed-zh-demos.ts`

5.2: RoundTable + AgentSeat + Bubble + AgentAvatar + PhaseBadge

- [ ] Build in `apps/web/app/room/[id]/components/v2/` isolated first
- [ ] Ellipse geometry (rx=280, ry=200), bubbles above, crossfade transitions
- [ ] Test with mock data for 6/9/12 agent counts

5.3: AgentDetailModal

- [ ] Click AgentSeat opens modal with model/persona/stats
- [ ] "View all messages from this agent" sub-view (filter by senderId)

5.4: ChatSidebar (WeChat-style)

- [ ] Right column 320px scrollable timeline
- [ ] Channel filter dropdown, mobile drawer via FAB

5.5: Wire both mode views

- [ ] Rewrite `RoundtableView.tsx` + `WerewolfView.tsx` to use v2 components
- [ ] Move legacy components to `components/legacy/` (kept for observability)

5.6: Polish + deploy

- [ ] Mobile breakpoints (375/768/1024px); ≤640px fallback strategy
- [ ] Reduced-motion + keyboard a11y (Esc, Tab)
- [ ] Vercel deploy + smoke-test en/zh × debate/werewolf/replay

Phase 6: Script Kill — Deep Narrative + Memory

Goal: Full murder mystery game with private clues, investigation phases, and branching narrative. Introduces long-term memory.

Step 6.1: Session Memory Compression (packages/core)

- [ ] `SessionMemory` class with LLM-powered compression
- [ ] When messages > threshold (e.g., 40), compress old messages into summary
- [ ] Keep summary + recent N messages in context window
- [ ] Use cheap model (Haiku) for compression

Step 6.2: Agent Long-Term Memory (packages/core)

- [ ] `AgentLongTermMemory` with pgvector
- [ ] record(event) → embed → store in Postgres
- [ ] retrieve(query) → cosine similarity search → inject into prompt
- [ ] reflect() — periodic self-reflection (Stanford Generative Agents pattern)

Step 6.3: Clue/Evidence System

- [ ] `Clue` type — content, visibility (who has seen it), importance
- [ ] Private clue distribution — each character gets unique clues at game start
- [ ] Clue discovery — agents can find new clues during investigation phases
- [ ] Clue sharing — agents can choose to share or withhold clues in discussion

Step 6.4: Script Kill Mode (packages/modes/script-kill)

- [ ] Script template system: define characters, clues, timeline, true culprit
- [ ] Phases: Introduction → Investigation Round 1-3 → Discussion → Accusation → Reveal
- [ ] Branching: investigation choices affect what clues are found
- [ ] AI script generation: given a theme, generate a complete murder mystery
- [ ] Deep persona prompts: each character has background, secrets, motivations

Acceptance Criteria (Phase 4): - [ ] Complete murder mystery runs with 4-6 agents - [ ] Each agent has unique private clues that others cannot see - [ ] Agents remember and reference earlier conversations (memory compression works) - [ ] At least one AI-generated script is playable and solvable - [ ] Agents with long-term memory show improved play over multiple games

Phase 7: TRPG — Open-World Narrative

Goal: AI Game Master runs a tabletop RPG session with AI/human players. Most complex mode.

Step 7.1: GM Agent

- [ ] Specialized agent type with world-state awareness
- [ ] Narrative generation: describe scenes, NPCs, consequences
- [ ] Rules engine integration: D&D 5e-lite skill checks
- [ ] Dynamic difficulty adjustment based on player engagement

Step 7.2: Dice System

- [ ] `DiceRoll` type — notation (d20, 2d6+3), result, context
- [ ] Skill check flow: player declares action → GM sets DC → roll → GM narrates outcome
- [ ] Visual dice animation in UI

Step 7.3: Character System

- [ ] Character sheet: name, class, stats, inventory, backstory
- [ ] Character creation wizard (AI-assisted)
- [ ] Character growth: XP, level up, new abilities
- [ ] Persistent characters across sessions

Step 7.4: TRPG Mode (packages/modes/trpg)

- [ ] Semi-freeform flow: GM-guided with player agency
- [ ] Combat mode (structured) vs Exploration mode (freeform)
- [ ] NPC management: GM creates and voices NPCs
- [ ] Campaign persistence: save world state between sessions

Acceptance Criteria (Phase 5): - [ ] GM Agent runs a coherent 30-60 minute adventure - [ ] Players can take creative actions (not just menu choices) - [ ] Dice rolls affect outcomes meaningfully - [ ] Campaign state persists across sessions - [ ] GM maintains world consistency (NPCs remember past interactions)

Phase 8: Platform — Open Up

Goal: Let users create their own modes. Build the ecosystem.

Step 8.1: Custom Mode SDK

- [ ] Mode definition schema (JSON/YAML for non-developers)
- [ ] Mode builder UI: define roles, flow, channels, prompts visually

- [ ] Mode validation: ensure mode definition is complete and consistent
- [ ] Mode testing sandbox: run a mode in preview before publishing

Step 8.2: Agent Marketplace

- [ ] Public persona gallery: browse and fork community-created agents
- [ ] Tags, ratings, usage counts
- [ ] Featured agents/modes curation

Step 8.3: Enhanced Replay

- [ ] Record all events with timestamps
- [ ] Replay viewer: step through a session like a recording
- [ ] Export as shareable link or video-like format
- [ ] Highlight reel: AI-generated summary of best moments

Step 8.4: Hierarchical FlowController

- [ ] Leader-worker pattern (三省六部 style)
- [ ] Task delegation and aggregation
- [ ] Review gates (propose → review → approve/reject)
- [ ] Multi-level hierarchy support

Acceptance Criteria (Phase 6): - [ ] Non-developer can create a custom mode via UI - [ ] At least 5 community-created modes published - [ ] Replay of any session is watchable and shareable - [ ] Hierarchical flow enables "OPC company" style simulations

Technical Milestones (Cross-Phase)

Milestone	Phase	Description
First agent speaks	1.2	AIAgent.reply() produces a message via Vercel AI SDK
First multi-agent conversation	1.2	3 agents in a Room exchange messages via FreeFormFlow
First UI render	1.5	Messages appear as chat bubbles in the browser
First deploy	1.6	Public URL serves a working roundtable debate
First info isolation	2.1	Wolf chat is invisible to villagers
First structured decision	2.3	Agent votes for a real player name (not hallucinated)
First game completion	2.4	Werewolf runs to win/loss without manual intervention

Milestone	Phase	Description
First human + AI game	3.3	Human plays alongside AI agents
First memory compression	4.1	Long conversation compressed without losing key facts
First cross-session memory	4.2	Agent references a previous game's events
First AI-generated content	4.4	AI writes a playable murder mystery script
First GM narration	5.1	GM Agent describes a scene and reacts to player input
First custom mode	6.1	User creates a mode without writing TypeScript

Risk Mitigation

Risk	Impact	Mitigation
LLM costs spiral during multi-agent sessions	High	Cost tracking per session, configurable limits, use cheaper models for compression/roles
Agents break character or ignore game rules	Medium	Structured output (Zod) for decisions, strong system prompts, few-shot examples
WebSocket scaling with many concurrent rooms	Medium	Socket.io rooms for isolation, consider PartyKit for edge if needed
Vercel AI SDK doesn't support a needed provider	Low	SDK has provider interface; can add custom providers
Session state loss on server restart	Medium	Persist room state to Postgres, implement session restore
Prompt injection by human players	Medium	Separate system prompt channel, input sanitization, rate limiting

Dependencies & Decisions Log

Decision	Made	Rationale
TypeScript full-stack (not Python)	2026-04-13	UI is core value prop; single-stack efficiency; Vercel AI SDK covers LLM needs
Not forking AgentScope	2026-04-13	v2.0 instability, dual-stack friction, Mode system absent, tool system covered by Vercel AI SDK
Turborepo monorepo	2026-04-13	Clean package boundaries, shared types, single dev command
Vercel AI SDK for LLM	2026-04-13	Multi-provider, streaming, structured output, tool use, actively maintained
Socket.io for realtime	2026-04-13	Mature, room-based broadcasting, reconnection, fallback to polling
Postgres + pgvector for storage	2026-04-13	Supabase hosting, vector search for long-term memory, no separate vector DB
Phase 1 = Roundtable (not Werewolf)	2026-04-13	Simplest mode validates platform core; market-validated by Accio Work demos